23-June-2026

Binary Search Algorithm

Q// find 36 In the sorted (ascending) array by using binary search!

0	1	2	3	4	5	6	7	8
2	4	6	9	11	12	14	20	36	48

arr. = [2, 4, 6, 9, 11, 12, 14, 20, 36, 48]

→ ascending

target = 36

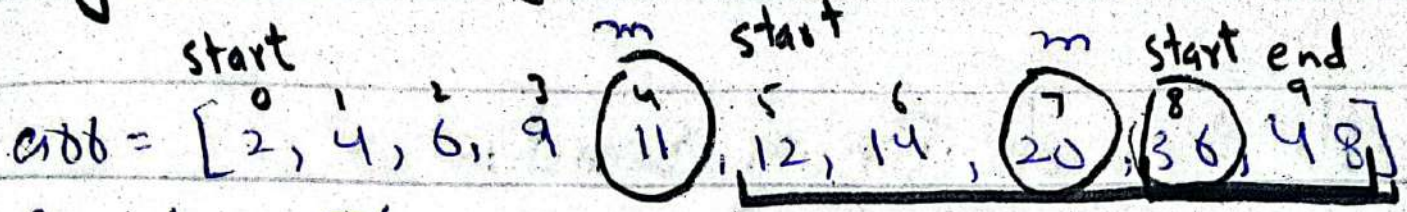
Steps::

1) find the middle index by total elements/2 (use int).
   ex: 9/5 = 4.5 or 4 so 4 index.
2) Imagine we know that our array is sorted ascending array so left side of middle index eg 4th index would have Possibility of 36 ? Absolutely not so we know that our target is on the right side of middle element.

if target > middle element => search in right side
else search in left hand side

3) if middle element == target element so ans.

Dry Run :- (using 3 steps)



target = 36

① middle element = $\frac{\text{start index} + \text{last index}}{2}$

$\frac{0+9}{2} = 4.5 = 4$ index

② 36 > 4 go to right hand side

now 12 becomes starts

① $\frac{5+9}{2} = \frac{14}{2} = 7$ index

② 36 > 20 go to right side

now 38 becomes start

① $\frac{8+9}{2} = \frac{17}{2} = 8.5 = 8$ index

② 36 > 36 ✗ → target arr[8] :: middle element

③ 36 == 36 ✓ ans = 8th index

if start > end => element not found

Best Case of Binary Search:-

↳ minimum steps occur.

When middle element equal to the targeted element.

Ex:

start

0 1 2 3 4 5 6 7 8 end

arr = [2, 4, 6, 9, 11, 12, 14, 20, 36, 48]

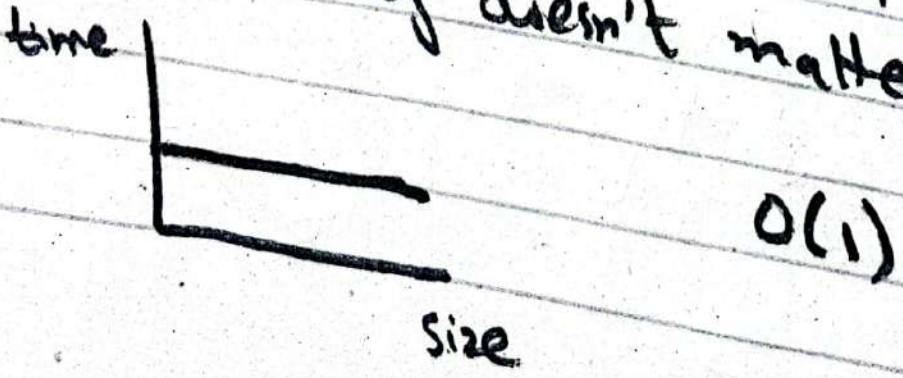
target = 11

$\frac{0+9}{2} \Rightarrow \frac{9}{2} \Rightarrow 4.5$ or 4

11 == 11 so return 4th index.

Best Case Complexity = Constant

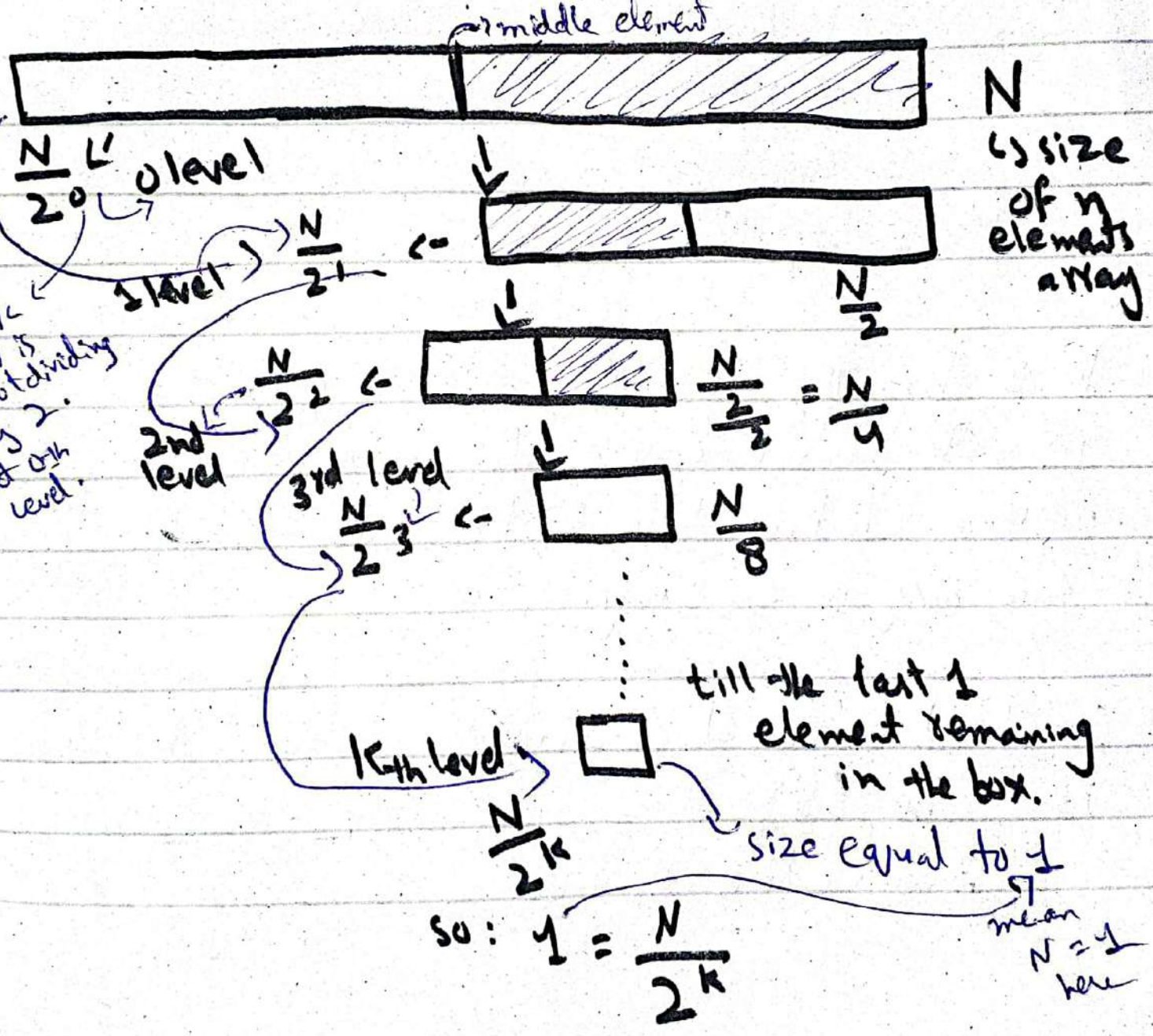
size of array doesn't matter.



Why Binary search?

Worst Case:-

Q// find the maximum number of such comparisons in the worst case.



So we got $\frac{N}{2^k} = 1$ or $N = 2^k$

k is total number of level.

for finding k (levels) use:

$$\log(N) = \log(2^k)$$

$$\log N = k \log 2$$

$$k = \frac{\log N}{\log 2}$$

$$\boxed{k = \log_2 N}$$

k is basically the total numbers of level in the worst case. b/c in worst case we are checking till the last of element remaining in an array.

So total number of comparisons in worst case is $\log(N)$.

∴ where N is the size of an array.

Total Comparisons in the worst case of binary search = $\log N$.

Q// Search in a sorted 1,000,000 size of array

Linear search will make 1,000,000 comparisons

Binary search will make maximum 20 comparisons

how?

by using the formula we derive

$\log_2(N)$

$\log_2(1000,000)$

$19.93 \approx 20$.

Complexity in Binary search is $O(\log N)$.

// better way to find middle number/ element

* $m = \frac{(start + end)}{2}$ → This may exceed the integer datatype limit (if array size is too long)

So, better way is:

* $m = \frac{(start + (end - start))}{2}$

$start + \frac{end - start}{2}$

$\frac{2\,start + end - start}{2}$

$\boxed{\frac{start + end}{2}}$ → same

Order agnostic Binary Search

```
          0   1   2   m   4   5   6   7
arr = [90, 75, 18, 12, 6, 4, 3, 1]
target = 75
```

descending order

target > middle element => left

end = mid - 1

target < middle element => right

start = mid + 1

Now, if we are giving a sorted array but don't know in Ascending or descending order so how we'll check?

By comparing 1st & last element of a given array.

Ex:

start end

arr = [1, 3, 3, 4, 14, 20, 33]

if (end > start) => increasing order array

else => descending order array

Q// why we dont compare 1st 2 elements to check whether its in ascending or descending order?

b/c

ex: s e -> Problem

arr = [3, 3, 3, 4, 8, 16, 33]

we know its ascending order array but if we compare first 2 elements so we dont find out. that's why we compare 1st & last element of an array